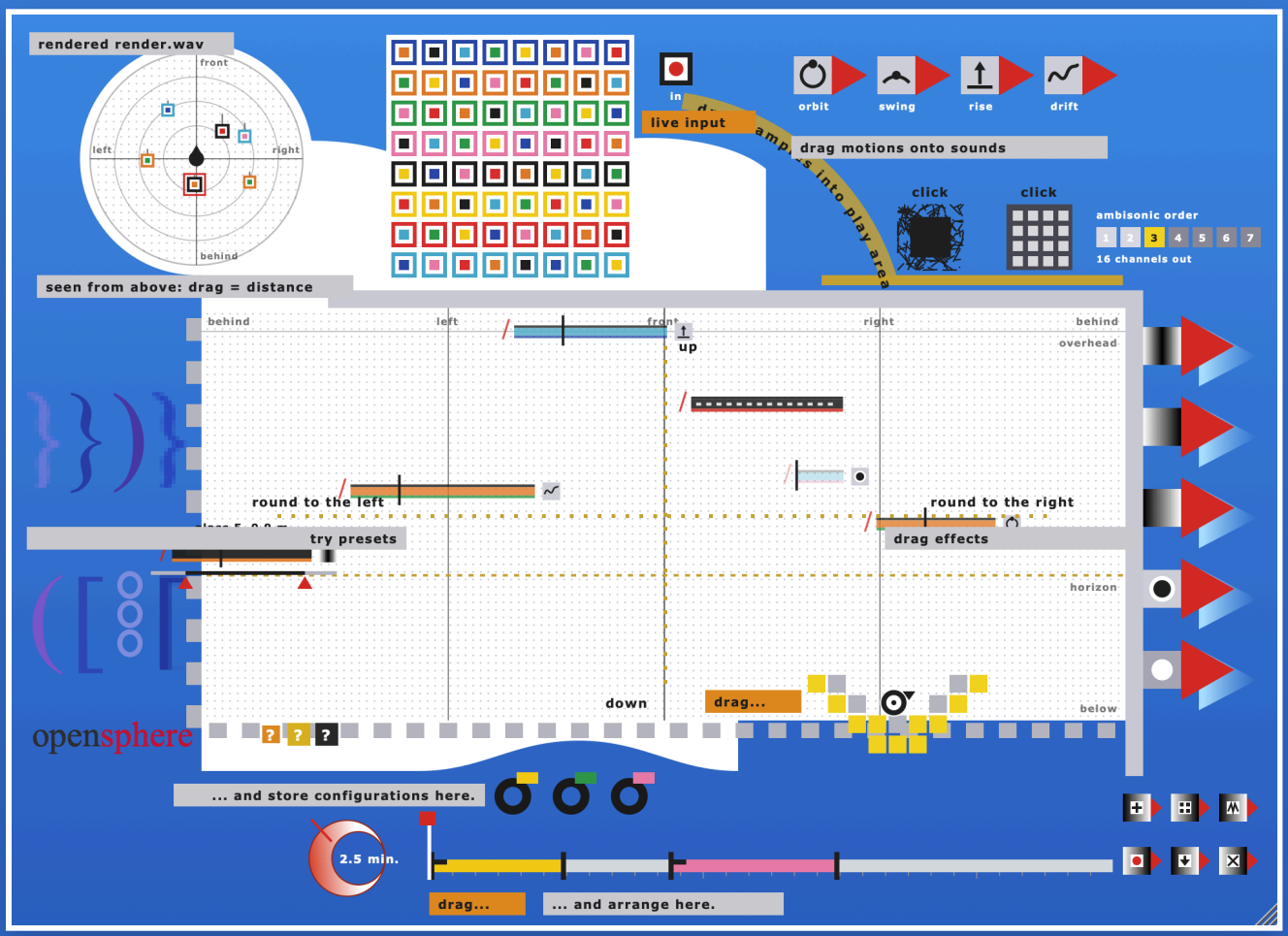
opensphere

manual · version 1.1

an ambisonics encoder



vst3 · vst2 · au · lv2 · standalone · macos · windows · linux

november 2026

0 contents

1 about opensphere
2 installing and setting up
3 the screen
4 the space
5 plan view and distance
6 motions
7 live input
8 output and order
9 browser and loops
10 effects
11 presets and logical operators
12 snapshots
13 timeline
14 render to file
15 sounds and tools
16 help and demo
17 midi and parameters
18 reference

1 about opensphere

opensphere is a loop workspace that places its sounds in a sphere around you. You drag sounds into a window, they loop, and you keep moving and changing them while they play. The output is Ambisonics, so the result can be decoded to any speaker ring or dome, or to headphones.

It is the twin of **openprocess**, a loop workspace modelled on **ovalprocess** (2000) by Markus Popp, who records as Oval. Both are built from the same source code. Everything openprocess does, opensphere does the same way: the browser with 64 loops, effects, presets, logical operators, snapshots, the timeline, your own sounds, rendering, MIDI. What changes is the play area and the output.

	openprocess	opensphere
play area left ↔ right	stereo panorama	direction, once round you
play area top ↔ bottom	loud ↔ soft	height, overhead ↔ 45° below
level	by height in the play area	by distance (plan view)
movement	you move the sounds	also four motions that move them by themselves
audio input	capture and Input Thru	capture, and a live input placed in the space
output	stereo	Ambisonics, 1st – 7th order (4 – 64 channels)

why loops in space

Oval's music is made of short loops that drift against each other and never fall into a common beat. In stereo they share two speakers and blur together. Spread round a listener, each loop keeps its own place: you can hear which fragment is where, follow one while the others go on, and turn your head to bring another forward. A loop that orbits or rises slowly changes the piece without any change to the sound itself. The rules of the original still hold: everything loops, every change is heard at once, and there are only symbols, no typing.

2 installing and setting up

opensphere comes in the **FreakiePie bundle** (with openprocess and Quell): one download per system from the releases page of `github.com/femifleming/freakiepie-bundle`. On a Mac, Homebrew does it in one line: `brew install --cask femifleming/tap/opensphere`.

system	put the files here
macOS	`opensphere.vst3` → `~/Library/Audio/Plug-Ins/VST3`, `opensphere.component` → `…/Components`, `opensphere.vst` → `…/VST`, `opensphere.app` → `/Applications`. Universal: Apple silicon and Intel.
Windows	`opensphere.vst3` → `C:\Program Files\Common Files\VST3`, `opensphere.dll` (VST2) → your VST folder; `opensphere.exe` runs on its own.
Linux	`opensphere.vst3` → `~/.vst3`, `opensphere.lv2` → `~/.lv2`, `opensphere.so` (VST2) → `~/.vst`; `opensphere` runs on its own (ALSA or JACK).

The macOS builds are signed ad hoc, not notarised. The Homebrew cask clears the quarantine flag; if you install by hand, right-click the app → Open the first time.

in a daw

opensphere needs a track with enough channels for the order you want: **4** for 1st order, **16** for 3rd, **36** for 5th, **64** for 7th (chapter 8). After it on the same track goes an Ambisonics **decoder**, which turns the channels into speaker feeds or binaural stereo for headphones.

1. Make a track and give it the channels. In **REAPER**: the track's *Routing* button → *Track channels* 16 (or 36, 64). The master track needs only 2 if the decoder is binaural.
2. Insert opensphere (VST3, VST2 or LV2). It offers up to 64 outputs and uses as many as the track has.
3. Insert a decoder after it, set to the same order and to **AmbiX (ACN / SN3D)**: for headphones IEM *BinauralDecoder* or SPARTA *AmbiBIN*; for speakers IEM *AllRADecoder* or SPARTA *AmbiDEC*.
4. Click a bracket on the left of opensphere. Something plays around you.

opensphere plays as soon as something is in the play area, whether or not the transport runs. It has nothing to do with the song tempo. Hosts that pass audio into the track feed the live input (chapter 7).

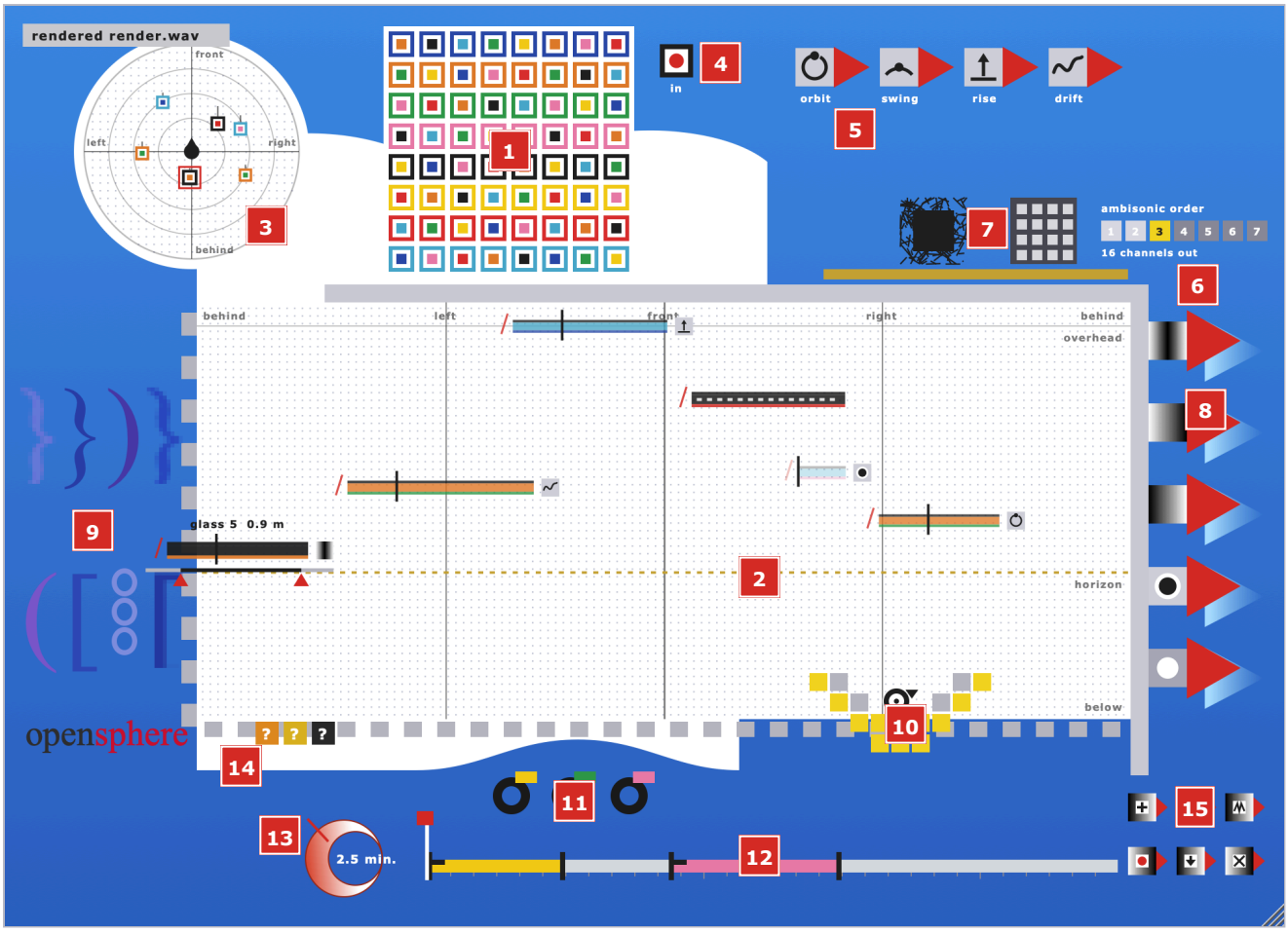
> **Ableton Live and Logic** have no tracks wide enough for this. There opensphere sees a stereo bus and plays a **stereo preview** (chapter 8). Build the piece with the preview and use *render to file* to write the full AmbiX file, which any Ambisonics player or decoder can then open. The **AU** version offers up to 3rd order (16 channels).

the standalone app

The app plays the stereo preview on the audio device set under *Options* → *Audio/MIDI Settings*; its renders are full AmbiX files at the chosen order. It asks for microphone access the first time, for capture and the live input. Your work is saved when you quit and is back when you start again.

The window can be resized from its lower right corner, from 600 to 1600 pixels wide. The proportions stay fixed.

3 the screen



1 browser: 8 × 8 loops

2 play area: the sphere around you, unrolled

3 plan view: the room seen from above, distance

4 live input (red **in** square)

5 motions: orbit, swing, rise, drift

6 ambisonic order and channels out

7 logical operators

8 effects

9 presets

10 snapshot button

11 snapshots (reels)

12 timeline with play flag

13 composition length wheel

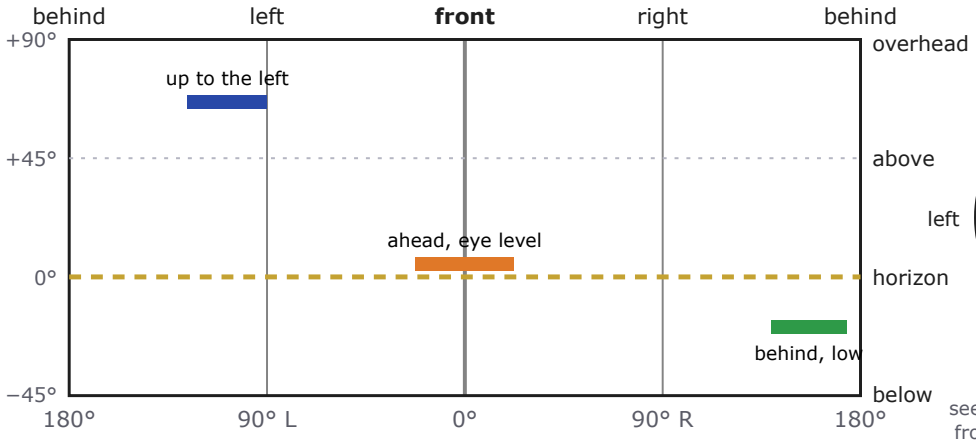
14 help marks · demo (word mark)

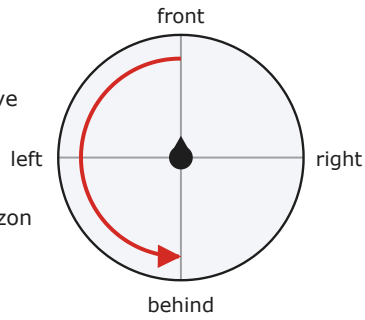
15 tools

In the picture, five loops and the live input (the bar with the white dashes) are playing. The selected sound, *glass 5*, shows its name, its distance (0.9 m) and the red pointers of its loop. Messages such as "rendered render.wav" or "the play area is full" appear for a few seconds in a grey tag at the top left.

4 the space

The play area is the sphere around you, cut open behind you and laid flat, like a map of the world. Where a sound sits in it is the direction it comes from.





seen from above: the play area from its middle to its left edge

left ↔ right: direction

The width of the play area is **once round you**. The middle line is straight **in front**. The quarter lines are **left** and **right**. Both edges are straight **behind**: they are the same place, where the sphere was cut. A sound that moves past one edge by itself (chapter 6) comes back in at the other.

The left half of the area is your left side, so the layout reads as if you faced the screen and the picture were wrapped round you.

top ↔ bottom: height

The height runs from **45° below** (bottom edge) to straight **overhead** (top). The dotted mustard line is the **horizon**, ear level, one third of the way up. Most of the area is above you, where a dome of speakers is; the lower part reaches below for spheres and headphones.

Near the top, directions close up: overhead every sound comes from the same point, whatever its left–right place. The height no longer changes the level, as it did in openprocess. Level comes from distance.

Sounds in the play area are drawn thick and solid when they are near and thin and faint when they are far away. Everything else about them works as in openprocess: drag to move, click to select, drag out to throw away.

5 plan view and distance



The circle at the top left is the room **seen from above**. You are the black head in the middle, nose to the front (up). Every sound in the play area appears here as a small square in its browser colours.

- The **angle** from you is the sound's direction, the same as its left–right place in the play area.
- The **distance** from you is how far away it is. The rings mark a quarter, half, three quarters and the whole of the range.
- A short **stalk** up or down shows that the sound is above or below the horizon.
- Far squares are drawn smaller. The selected sound has a red frame.

Drag a square in the plan view to set direction and distance at once. Its height stays as it was.

Turn the mouse wheel over a sound, in the plan view or in the play area: up or away brings it nearer, down or towards you sends it further. The sound is selected and its distance is shown in metres above its bar.

what distance changes

Distance runs from 0 (at your head) to 1 (the outer ring). It is shown in metres as **0.25 + 19.75 · d²**, so from 25 cm to 20 m. A new sound starts at about 2 m. Four things follow from it:

- **level**: 0 dB at your head, about −6 dB at 3 m, about −29 dB at the far edge.
- **width**: the two channels of a stereo loop are encoded as two points either side of its direction, 80° apart when near, 10° apart (almost a point) when far.
- **brightness**: a low-pass filter, the air, closes from 18 kHz near you to 3.5 kHz at the far edge.
- **near field**: within the innermost 0.12 (about half a metre) the higher orders fade out, so the sound loses its direction and surrounds you. At distance 0 it is inside your head.

distance	metres	level	width	air	direction
0	0.25	0 dB	80°	18 kHz	none, all round you
0.12	0.53	−1.7 dB	72°	17.6 kHz	full from here on
0.3	2.03	−4.6 dB	59°	15.5 kHz	a new sound
0.5	5.19	−8.7 dB	45°	12 kHz	
0.75	11.4	−15.9 dB	28°	7.2 kHz	
1	20	−28.8 dB	10°	3.5 kHz	the outer ring

All gains glide over about 20 ms, so dragging and moving sounds never click. Distance is not a delay: there is no Doppler shift and no echo of a room. Add a reverb after the decoder if you want one. Unlike openprocess, nothing is silent by position: to silence a sound, mute it or throw it away.

6 motions



The four icons on red pointers at the top right are **motions**. They work like effects: **drag one onto a sound** in the play area and it sticks to the sound as a small icon after its bar (after the effect icon, if it has one). From then on the sound moves through the space by itself. A sound carries one motion at a time and one effect at a time; the two are independent.

motion	what the sound does	period
orbit	goes once all the way round you, turning to the left: front, left, behind, right, front. Its height and distance stay the same. All orbiting sounds turn together and keep their spacing.	24 s
swing	swings to either side of its place, ±40°, like a pendulum.	8 s
rise	rises from its height up to straight overhead and sinks back, smoothly.	16 s
drift	wanders slowly round its place, sideways and up and down, a few tens of degrees, never quite repeating.	slow, irregular

- **To remove a motion**, drag its icon off the sound, or drop the same motion on it again. Dropping the icon on another sound moves the motion there.
- The place where you put the sound is its **resting place**. The motion is added to it. Drag a moving sound and you move its resting place; it carries on moving from where you let go.
- Swing, rise and drift start at a different point of their cycle for each sound, so two sounds with the same motion do not move in step.
- Moving sounds move in the plan view too. Motions are stored in snapshots. On the timeline and in renders they follow the composition's clock, so a piece moves the same way every time it plays.

7 live input



The red square marked **in**, right of the browser, is the audio input of the plug-in (or the app's audio device). Its dot grows with the input level. **Drag it into the play area** and the input is placed in the space like a loop: a black bar with white dashes, whose red line shows the level. It does not loop; you hear the input as it comes in.

- Move it, set its distance in the plan view or with the mouse wheel, and give it a motion, as with any sound. A stereo input is spread by distance like a stereo loop; a mono input feeds both sides.
- Of the effects only **mute** and **solo** act on it; fades follow a loop, and the input has none. It has no loop pointers.
- It can be placed more than once, for example once near and once far, or once still and once orbiting.
- It is stored in snapshots and plays on the timeline. A **render** has no input, so there it is silent: record the input as a loop first with *capture input* (chapter 15) if it should be in the file.

8 output and order



opensphere's output is **Ambisonics**: not speaker feeds but a description of the whole sound field, which a decoder turns into whatever you listen on. The format is **AmbiX**: channels in **ACN** order with **SN3D** normalisation, as used by the IEM and SPARTA plug-ins, YouTube and most players.

The seven squares at the top right choose the **order**. Click one. The chosen order is yellow; the orders actually being sent are light. Below it, "**N channels out**" tells what goes out, or "stereo preview".

order	channels	where
1st	4	all formats
2nd	9	all formats
3rd	16	all formats; default; AU maximum
4th	25	VST3, VST2, LV2
5th	36	VST3, VST2, LV2
6th	49	VST3, VST2, LV2
7th	64	VST3, VST2, LV2

Channels = (order + 1)². Renders are possible at every order in every format, the app included.

which order

Higher orders place sounds more sharply but need more channels and more speakers to pay off. **1st order** is enough for a rough image and for many 360° video players. **3rd order** is a good default for headphones and small rings. 5th to 7th suit large domes.

If the track has fewer channels than the chosen order needs, opensphere sends the highest order that fits, and the label shows it: with 16 channels and 5th order chosen, 3rd order goes out.

stereo preview

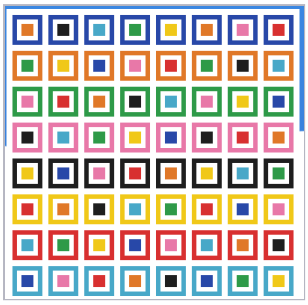
On a stereo bus (Ableton Live, Logic, any 2-channel track) and in the standalone app, opensphere encodes at 1st order and folds it to left and right: **W + Y** and **W − Y**, two virtual microphones pointing left and right. You hear where sounds are to the left or right, their distance, width and motion. You do not hear height, and front and back sound the same. The preview has a soft limiter; the Ambisonic output has none, so keep *Master* down when many near sounds pile up.

decoding

Put the decoder directly after opensphere on the same track. Set it to the same order and to ACN / SN3D. A binaural decoder gives headphone stereo; an AllRAD or similar decoder needs your speaker positions. If the decoder can turn the sound field (or a head tracker can), a separate rotator plug-in such as IEM *SceneRotator* goes between the two.

> Hosts treat wide tracks differently. If you hear nothing, check that the track has at least 4 channels, that the decoder reads the same number, and that its output goes to the master. In a stereo-only host the label reads "stereo preview".

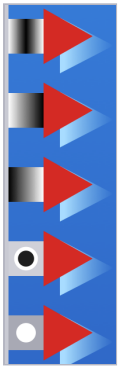
9 browser and loops



The browser holds 64 loops in an 8 × 8 grid. The outer colour of a square shows its **row**, the family of sounds; the inner colour tells the columns apart. **Drag a square into the play area** and the loop starts at that place, 2 m away. **Click a square** to select it for *skip again* and *capture input*. A pale square is still being made.

Click a sound in the play area to select it. Below it a grey strip stands for the whole sample, with **two red pointers**: the part between them loops. Loop ends are crossfaded over about 6 ms. Up to **24 sounds**, the live input included, can play at once.

10 effects

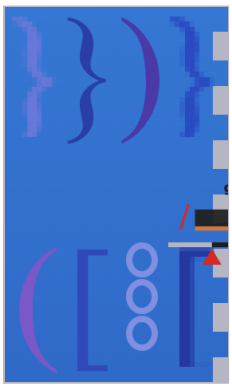


The five effects on the right edge work as in openprocess. **Drag one onto a sound**; it sticks as a small icon after the bar. One effect per sound; a new one replaces the old.

icon (top to bottom)	effect
black in the middle	**fade in / out** over each loop
white → black	**fade in** over each loop
black → white	**fade out** over each loop
black dot	**mute**: the sound keeps running, silently
white dot	**solo**: only sounds marked solo are heard

To remove an effect, drag its icon off the sound or drop the same effect on it again. Dropping it on another sound moves it there.

11 presets and logical operators



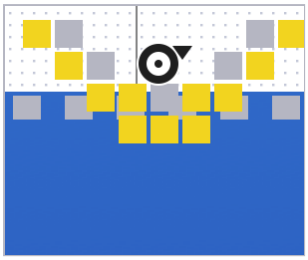
The eight brackets on the left are **presets**. A click fills the play area with 3 to 5 sounds, mostly from one browser row, never the same twice. Odd brackets use long parts of the samples (40–100 %), even brackets tiny fragments (3–25 %). **Shift-click** adds instead of replacing.

In opensphere, preset sounds are spread **all round you**, mostly a little above the horizon (from 15° below to 40° above), between half a metre and about 10 m away. Each has a one-in-four chance of a fade and about a one-in-three chance of a motion.

The two **logical operators** above the play area change everything at once:

- **black scribble**: new loop lengths and start points for every sound, from 2 % to the whole sample.
- **grey grid**: a new place for every sound: any direction, from 30° below to 80° above, and a new distance. They glide there; motions and effects stay.

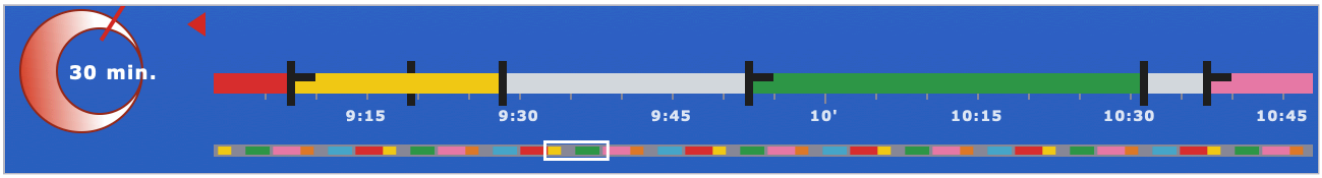
12 snapshots



Click the yellow chequered **snapshot button** at the bottom of the play area and the whole scene is stored: every sound with its direction, height, distance, loop, effect and motion, and any live input. The snapshot lines up below as a **reel**, a black ring with a colour tag. There is room for **10 reels**.

- **Drag a reel onto the timeline** to use it in a composition.
- **Drag a reel back into the play area** to hear that scene again. It replaces what is there.
- **Throw a reel out of the window** to delete it.

13 timeline



Reels dropped on the timeline become **segments** in the reel's colour: 20 seconds long, or an eighth of the stretch on screen when you are zoomed out.

- **drag** a segment to move it, **drag its right end** to change its length
- **alt-drag** to copy it, **drag it away** (off the bar) to delete it

Overlapping segments play together. Each keeps its own copy of the snapshot. Segments fade in and out over 30 ms. There is room for **256 segments**.

wheel

The red crescent sets the **length of the composition, from 1 to 30 minutes**: drag up or right for longer. The scale is logarithmic, in tenths of a minute at first and whole minutes above ten. Normally segments keep their places relative to the whole and stretch with it. **Shift-drag the wheel** and they keep their times in seconds instead; only the end moves (segments past a shorter end are dropped).

zoom

Thirty minutes do not fit on 540 pixels, so the timeline zooms. Over the timeline:

- **mouse wheel** up / away, or **pinch**: closer in, around the pointer
- **sideways scroll**, or **shift + wheel**: along the piece
- **double-click** the bare bar: 4× closer, or back to the whole piece

Zoomed in, a thin **overview strip** under the bar shows the whole piece with a white window round the part on screen; drag the window or click the strip to jump. Ticks are written as `9:30` or `10'`. You can zoom in until ten seconds fill the bar. Above: 30 minutes, zoomed in on 9–11 min.

play flag

The **red flag** is the play pointer. **Click it** to play from the flag to the end and round again, until you click again. **Drag it** to start later. Off screen, a red arrow shows where it is; while playing, the view pages along. **While it plays nothing else can be changed** (zoom still works); motions keep moving.

14 render to file

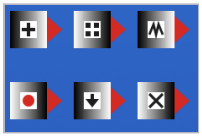
render to file (tools, ↓) writes the piece as an **AmbiX WAV**: 48 kHz, 24-bit, ACN / SN3D, with as many channels as the chosen order has, whatever the host or the app is playing. If the timeline has segments, the render runs from the play flag to the end of the composition; if not, it records the play area for the length set on the wheel. It is mixed at the current Master level.

The file is written as it goes, so a long piece needs no more memory than a short one; the corner counts up the percent. The default place is `~/Music/opensphere` (your Music folder on Windows and Linux too), with a name like `opensphere 2026-11-13 21.04.10.wav`. Files over 4 GB are written as **RF64**, the WAV variant without that limit; most audio editors and Ambisonics players read it.

order	channels	per minute	30 minutes
1st	4	35 MB	1.0 GB
3rd	16	138 MB	4.1 GB
5th	36	311 MB	9.3 GB
7th	64	553 MB	16.6 GB

The live input is not part of a render (chapter 7). Motions follow the composition's clock, so the same piece renders the same way each time. A 30-minute piece at 3rd order is just under 4 GB, an ordinary WAV; at 4th order and above it becomes RF64.

15 sounds and tools



button	does
+ load sounds	choose files or folders to chop into the browser
▦ built-in sounds	restore the synthesized bank
^^ skip again	re-chop the selected square with a new pattern
• capture input	the last 8 s of input, silence trimmed, into the selected square; saved in `~/Music/opensphere/captures`
↓ render to file	write an AmbiX WAV (chapter 14)
× clear	empty the play area

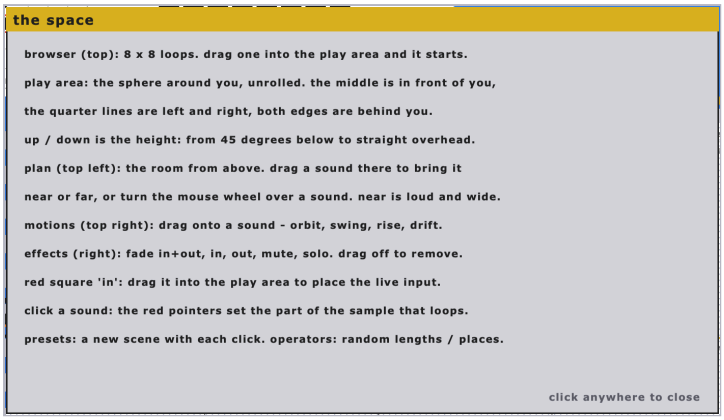
the built-in bank

All 64 loops are made by opensphere when it starts, the same bank as openprocess. Rows: **organ**, **guitar**, **feedback**, **skips**, **glass**, **low**, **noise**, **skipped**. All use the same pitches (D, E, F♯, A, B, C♯), so any can be combined. Loops are chopped like a skipping CD. **Columns run from clean on the left to heavily skipped on the right.** Loops are 0.5 to 9 seconds long and none is a multiple of another, so they drift against each other.

your own sounds

- **Drop audio files or folders onto the browser** (or use *load sounds*). Up to 256 files are chopped into all 64 squares.
- **Drop a single file onto one square** to replace only that square (first 30 seconds, unchopped).
- Formats: WAV, AIFF, FLAC, Ogg Vorbis, MP3, plus M4A/CAF on the Mac. A session remembers where its files are, not the audio.

16 help and demo



The three **?** marks next to the word mark open help without stopping the sound:

- **?** orange: the **guide**, the labels on the cover of this manual ("seen from above: drag = distance", "drag motions onto sounds", "live input", "round to the left" …). Click again to hide them.
- **?** yellow: **the space** (above): the play area, plan view, motions, effects and live input.
- **?** black: **snapshots, timeline, output**: reels, the timeline and wheel, the Ambisonic output, decoders and the stereo preview, render and MIDI.

Click the word mark to start the **demo**. A cursor works through the functions on its own: it drags sounds, applies effects, uses presets and operators, takes snapshots, arranges and plays them. **Any click ends the demo** and puts back exactly what you had. The standalone app starts the demo by itself after three minutes without input when the play area is empty.

17 midi and parameters

note	number	action
C1 – G1	36 – 43	presets 1 – 8
G♯1	44	operator: lengths
A1	45	operator: places
A♯1	46	snapshot
B1	47	timeline play / stop
C2 – D♯7	48 – 111	browser squares 1 – 64: add, or remove if already playing

Middle C = C3. Notes act on note-on; velocity and note-off are ignored. While the timeline plays, only B1 has an effect. A square added by MIDI goes to a random direction, from 10° below to 40° above, 2 m away.

parameter	range	default
Master	0 – 1	0.8
Input Thru	0 – 1	0
Ambisonic Order	1st – 7th	3rd

Master scales everything that goes out and into renders. **Ambisonic Order** is the same as the seven squares; it can be automated, though a decoder rarely follows a change of order mid-piece.

Input Thru is kept so that sessions and automation match openprocess, but opensphere ignores it: the input is heard only where you place it (chapter 7).

18 reference

item	value
browser	64 loops (8 rows × 8 columns), 0.5 – 9 s each
sounds in the play area	up to 24, live inputs included
direction	play area left → right: behind, left, front, right, behind (once round you)
height	play area bottom → top: −45° to +90°; horizon one third up
distance	0 – 1, shown as 0.25 – 20 m (0.25 + 19.75 · d²); new sounds 2 m
level by distance	0 dB at your head, −6 dB at about 3 m, −29 dB at 20 m
width of a stereo sound	80° near → 10° far
air	low-pass 18 kHz near → 3.5 kHz far
near field	higher orders fade below distance 0.12 (about 0.5 m)
gain smoothing	about 20 ms
motions	orbit 24 s once round · swing ±40°, 8 s · rise to overhead and back, 16 s · drift slow wander
output format	Ambisonics, ACN / SN3D (AmbiX), no Condon–Shortley phase
order	1st – 7th (4 – 64 channels), default 3rd; AU up to 3rd (16)
stereo preview	1st order folded to W + Y / W − Y; stereo buses and the standalone app
input	stereo or mono; capture (last 8 s) and the live input
snapshots	up to 10
timeline segments	up to 256; new segments 20 s
composition length	1 – 30 minutes; the timeline zooms to 10 s
render	WAV, 48 kHz, 24-bit, AmbiX at the chosen order, streamed to disk, RF64 over 4 GB
user folder	`~/Music/opensphere`
saved with the session	play area (with distances and motions), snapshots, timeline, length, flag, how each square was made, parameters
window	1000 × 720 at 100 %, resizable 60 – 160 %
plug-in code	`OpSp` (openprocess and opensphere can be used side by side)

Decoders that read opensphere directly: IEM Plug-in Suite (BinauralDecoder, AllRADecoder, SimpleDecoder, SceneRotator, up to 7th order) · SPARTA (AmbiBIN, AmbiDEC, Rotator) · any player or decoder that reads AmbiX (ACN / SN3D) files.

sources

Ovalprocess sound installation, Dokumentation, Bedienungs- und Wartungshinweise, and Markus Popp, ovalprocess project text (Berlin, 7 April 2001), Ars Electronica archive · Markus Popp, ovalprocess US 2k lecture v0.2, Ars Electronica archive · Oval, "Ovalprocess", *Leonardo* 35/5, 2002 · Sam Inglis, "Markus Popp: Music As Software", *Sound On Sound*, October 2002 · Marc Weidenbaum, interview with Markus Popp, Disquiet / Pulse!, 1996.
opensphere is an independent homage. It is not made, endorsed or distributed by Markus Popp or Oval.